



EcoRide

Plateforme de covoiturage écologique

Évan OROFINO

TP — Développeur Web et Web Mobile

Studi · Promotion 2026

ECF · CCP1 — Hiver 2025 / Printemps 2026

Dépôt GitHub public : github.com/EvanOROFINO/ecoride · 6 mai 2026

Sommaire

1. Manuel d'utilisation
2. Charte graphique
3. Documentation technique
4. Modèle Conceptuel de Données
5. Schéma MongoDB
6. Gestion de projet
7. Kanban

1

Manuel d'utilisation

Manuel d'utilisation — EcoRide

Plateforme de covoiturage écologique — Version 1.0

1. Présentation

EcoRide est une plateforme de covoiturage qui privilégie les trajets écologiques (voitures électriques) et facilite le partage entre conducteurs et passagers. Le site est accessible à trois types d'utilisateurs : **visiteurs**, **utilisateurs** (chauffeurs et/ou passagers), **employés** et **administrateurs**.

2. Accès à l'application

- **En local** : <http://localhost:8000> (serveur PHP)
- **En production** : <https://ecoride.up.railway.app> (à compléter après déploiement)

3. Comptes de démonstration

Mot de passe pour tous les comptes : **Password123!**

Rôle	Email	Pseudo	Crédits
Administrateur	admin@ecoride.fr	admin	9999
Employé	employe@ecoride.fr	employe1	100
Chauffeur + passager	sophie@example.com	sophie_b	50
Chauffeur	lucas@example.com	lucas_m	75
Chauffeur + passager	julie@example.com	julie_r	40
Passager	tom@example.com	tom_d	20
Passager	emma@example.com	emma_l	30
Passager	paul@example.com	paul_g	25

4. Parcours visiteur

4.1 Page d'accueil (US 1)

1. Ouvrir <http://localhost:8000>
2. La page présente l'entreprise, une barre de recherche et un footer (mail + mentions légales)

4.2 Recherche d'un covoiturage (US 3-4)

1. Sur la page d'accueil, saisir une ville de départ (**Paris**), d'arrivée (**Lyon**) et une date
2. Cliquer sur **Rechercher**
3. La liste des trajets correspondants s'affiche
4. Utiliser les filtres dans la sidebar pour affiner :
 - **Écologique uniquement** : ne montre que les voitures électriques
 - **Prix maximum** : en crédits par personne
 - **Durée maximum** : en minutes
 - **Note minimale** : filtre par note moyenne du chauffeur
5. Si aucun résultat, le site propose la **prochaine date disponible**

4.3 Détail d'un covoiturage (US 5)

1. Cliquer sur **Détails** d'un trajet
2. La page affiche : véhicule, marque, énergie, préférences chauffeur, avis validés
3. Bouton **Participer** si connecté avec assez de crédits

5. Parcours utilisateur (US 6, 7, 8, 9, 10, 11)

5.1 Création de compte (US 7)

1. Cliquer sur **Connexion** > **Créer un compte**
2. Renseigner un pseudo (3-50 car.), un email valide, un mot de passe sécurisé
 - **Min 8 caractères**
 - **1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial**
3. À la création, le compte reçoit **20 crédits offerts**

5.2 Espace utilisateur (US 8)

Une fois connecté, accéder à **Mon espace** :

- **Choisir son rôle** : chauffeur, passager, ou les deux (case à cocher)
- **Mes véhicules** (chauffeur uniquement) : ajouter une voiture (marque, modèle, immatriculation, énergie, places)
- **Mes préférences** : fumeur, animaux, et préférences personnalisées (clé/valeur libre stockées en MongoDB)

5.3 Proposer un voyage (US 9)

1. **Mon espace** > **Proposer un voyage**
2. Renseigner : départ, arrivée, dates, heures, véhicule, places, prix
3. Le prix doit être ≥ 3 crédits (2 prélevés par la plateforme)

5.4 Participer à un voyage (US 6)

1. Sur la page détail d'un trajet, cliquer **Participer**

2. **Double confirmation** demandée
3. Les crédits sont prélevés immédiatement, la place décrémentée

5.5 Historique (US 10)

- **Mon espace > Mon historique**
- Visualiser tous ses trajets (chauffeur et passager)
- **Annuler** un trajet (chauffeur : remboursement passagers + mail) ou (passager : remboursement de ses crédits)

5.6 Démarrer / arrêter un trajet (US 11)

- **Chauffeur** : depuis l'historique, bouton **Démarrer** puis **Arrivée à destination**
- **Passager** : après l'arrivée, bouton **Valider le trajet**
 - Indiquer si tout s'est bien passé (oui/non)
 - Si oui : noter (1-5 étoiles) + commentaire — l'avis passe en modération employé
 - Si non : commentaire requis — un employé contactera le chauffeur

6. Parcours employé (US 12)

Connexion avec `employe@ecoride.fr` . Accéder à **Espace employé** :

6.1 Modération des avis

- **Espace employé > Avis à modérer**
- Liste des avis en attente
- **Valider** ou **Refuser** chaque avis (le chauffeur ne voit que les avis validés)

6.2 Gestion des incidents

- **Espace employé > Incidents**
- Liste des trajets signalés problématiques
- Coordonnées chauffeur et passager affichées (mail cliquable)
- Description du problème pour traitement

7. Parcours administrateur (US 13)

Connexion avec `admin@ecoride.fr` . Accéder à **Admin** :

7.1 Tableau de bord

- 3 KPI : crédits gagnés, utilisateurs, covoiturages
- **Graphique** : nombre de covoiturages par jour (30 derniers jours)
- **Graphique** : crédits gagnés par jour (30 derniers jours)

7.2 Gestion des comptes

- **Admin > Gérer les comptes**
- Liste de tous les utilisateurs avec rôles
- **Suspendre** ou **Réactiver** un compte (utilisateur ou employé)
- **Créer un compte employé** depuis le formulaire dédié

Note : la création du compte administrateur se fait directement en BDD (script SQL `02_seed.sql`), pas depuis l'application.

8. Sécurité

- Mots de passe hashés en **bcrypt** (jamais stockés en clair)
- **Protection CSRF** sur tous les formulaires POST
- **Sessions HTTPOnly** + `SameSite=Lax` + régénération à la connexion
- **Requêtes préparées PDO** (protection injection SQL)
- **Échappement HTML** (htmlspecialchars) sur tous les affichages utilisateur (anti-XSS)
- En-têtes de sécurité : `X-Content-Type-Options` , `X-Frame-Options` , `Referrer-Policy`

9. Support

- Email : `contact@ecoride.fr`
- Page contact : `/contact`
- Mentions légales : `/mentions-legales`

2

Charte graphique

Charte graphique — EcoRide

1. Concept

L'identité visuelle d'EcoRide s'inspire de l'écologie et du voyage. Les couleurs évoquent la nature (vert pour le végétal, bleu pour le ciel et l'eau, beige pour la terre). Le ton est moderne, accessible et rassurant.

2. Palette de couleurs

Rôle	Nom	HEX	RGB	Usage
Primaire	Vert forêt	#2E7D32	46, 125, 50	Boutons principaux, liens actifs, header
Primaire clair	Vert tendre	#66BB6A	102, 187, 106	Hover, accents, badges écologiques
Secondaire	Bleu ciel	#0288D1	2, 136, 209	Liens, info, icônes
Accent	Ocre / Sable	#F9A825	249, 168, 37	CTA secondaires, étoiles de notation
Fond clair	Blanc cassé	#F5F7F4	245, 247, 244	Fond principal des pages
Fond carte	Blanc pur	#FFFFFF	255, 255, 255	Fond des cartes/encarts
Texte principal	Anthracite	#212121	33, 33, 33	Corps de texte
Texte secondaire	Gris moyen	#616161	97, 97, 97	Labels, métadonnées
Erreur	Rouge brique	#C62828	198, 40, 40	Messages d'erreur
Succès	Vert validé	#388E3C	56, 142, 60	Messages de succès

3. Typographie

- **Titres (h1-h3)** : **Poppins**, sans-serif, font-weight 600
- **Sous-titres (h4-h6)** : **Poppins**, sans-serif, font-weight 500
- **Corps de texte** : **Inter**, sans-serif, font-weight 400, taille 16px, line-height 1.6
- **Boutons** : **Poppins**, sans-serif, font-weight 600, taille 14px, lettres en majuscules

Polices chargées via Google Fonts :

```
<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700&family=Inter:wg
```

4. Espacement et grilles

- Système de grille Bootstrap 5 (12 colonnes)
- Espacement de base : multiples de 8px (8, 16, 24, 32, 48, 64)

- Border-radius standard : 8px (cartes), 4px (champs de formulaire), 24px (boutons pilule)

5. Composants

Boutons

- **Primaire** : fond #2E7D32 , texte blanc, hover #1B5E20
- **Secondaire** : fond transparent, bordure #2E7D32 , texte #2E7D32 , hover fond #E8F5E9
- **Danger** : fond #C62828 , texte blanc

Cartes covoiturage

- Fond blanc, ombre légère 0 2px 8px rgba(0,0,0,0.08)
- Border-radius 8px, padding 16px
- Badge "Écologique" : fond #66BB6A , texte blanc, icône feuille

Formulaires

- Champs : bordure #BDBDBD , focus bordure #2E7D32 , padding 12px 16px
- Labels : couleur #616161 , taille 14px, font-weight 500

6. Iconographie

- Bibliothèque : **Bootstrap Icons** (cohérence avec Bootstrap 5)
- Icônes thématiques : feuille (écologique), voiture, étoile (notation), euro (prix), horloge (durée)

7. Logo

Logo textuel : "EcoRide" en Poppins 700, avec une icône feuille à gauche du nom.

- Couleur principale : #2E7D32
- Variante sur fond sombre : blanc avec icône #66BB6A

8. Accessibilité

- Contraste minimal AA (WCAG 2.1) sur tous les textes
- Taille de texte minimale : 14px
- Focus visible sur tous les éléments interactifs
- alt obligatoire sur toutes les images informatives

3

Documentation technique

Documentation technique — EcoRide

TP Développeur Web et Web Mobile (Studi) — Auteur : Evan Orofino — 2026

1. Réflexions initiales et choix technologiques

1.1 Contexte

EcoRide est une plateforme de covoiturage écologique avec 13 user stories à implémenter, devant gérer :

- Une **base relationnelle** (utilisateurs, voitures, trajets, participations)
- Une **base NoSQL** (avis, préférences, configuration)
- Un **front-end responsive**
- Un **back-end sécurisé**
- Un **déploiement** documenté

1.2 Stack retenue

Couche	Choix	Pourquoi
Front	HTML5 + CSS3 + JS vanilla	Pas de framework lourd, contrôle total, performances maximales, simple à défendre à l'oral
CSS	Custom + variables CSS	Charte graphique propre sans dépendance, mobile-first
Icons	Bootstrap Icons (CDN)	Léger, cohérent, large catalogue
Charts	Chart.js (CDN)	Standard de fait, simple à intégrer
Back	PHP 8.2 + PDO	Recommandé Studi, large communauté, déploiement universel
BDD relationnelle	MySQL 8 / MariaDB 10	Mature, performant, bien intégré à PHP
BDD NoSQL	MongoDB	Schéma flexible idéal pour avis et préférences
Templating	PHP natif (vues)	Pas de couche supplémentaire à apprendre/défendre
Autoloading	Composer PSR-4	Standard PHP, classes auto-chargées
Déploiement	Railway.app	Plan gratuit, déploiement Git-push

1.3 Alternatives considérées et écartées

Alternative	Pourquoi écartée
Framework Symfony / Laravel	Surdimensionné pour 13 US, ajoute une couche d'apprentissage masquant les concepts
Node.js / Express	Possible, mais PHP est explicitement mentionné dans la consigne et plus aligné avec la formation DWWM
React / Vue côté front	Inutile pour un site avec 90 % de contenu statique côté serveur ; SEO et perfs initiales meilleures avec SSR PHP
Bootstrap CSS	Custom CSS plus léger et cohérent avec la charte ; on garde Bootstrap Icons uniquement

2. Configuration de l'environnement

2.1 Prérequis

- Windows 10/11 (testé) ou Linux/macOS
- XAMPP 8.2 (ou PHP 8.2 + MySQL 8 séparés)
- Composer 2.x
- Git 2.x
- Extension PHP **mongodb** (driver PECL)
- Compte GitHub (dépôt public)
- Compte MongoDB Atlas (cluster gratuit M0)

2.2 Installation pas à pas

```
# 1. Cloner le dépôt
git clone https://github.com/<votre-user>/ecoride.git
cd ecoride

# 2. Configurer l'environnement
cp .env.example .env
# Éditer .env : DB_*, MONGO_URI, APP_SECRET

# 3. Installer les dépendances PHP
composer install

# 4. Créer et peupler MySQL
mysql -u root < sql/01_schema.sql
mysql -u root < sql/02_seed.sql

# 5. Initialiser MongoDB
php sql/init_mongo.php

# 6. Lancer le serveur de dev
php -S localhost:8000 -t public
```

2.3 Structure du projet

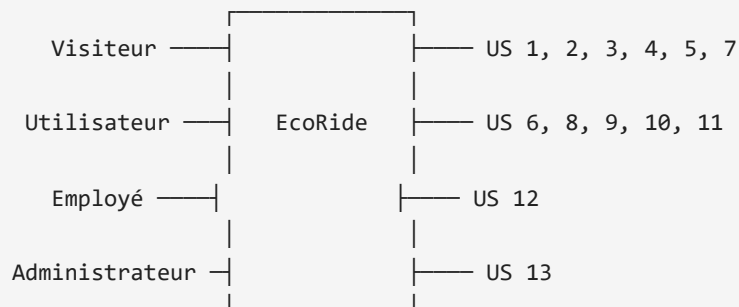
```
ecoride/
├── config/                # Configuration (BDD, Mongo, app)
│   └── config.php
├── public/               # Document root (servi par Apache/PHP)
│   ├── index.php        # Front controller (toutes les requêtes)
│   ├── .htaccess        # URL rewriting + headers sécurité
│   ├── css/app.css
│   ├── js/app.js
│   └── images/
├── src/                  # Code applicatif (autoload PSR-4 → App\ )
│   ├── Core/            # Database, Mongo, Router, Controller, Auth, Security
│   ├── Controllers/     # Logique HTTP (par domaine fonctionnel)
│   ├── Models/          # Accès aux données SQL et Mongo
│   └── Helpers/functions.php
├── views/                # Templates PHP (un dossier par domaine)
│   ├── layouts/main.php
│   ├── partials/
│   ├── home/, auth/, covoiturage/, user/, employe/, admin/, errors/
├── sql/
│   ├── 01_schema.sql    # DDL MySQL
│   ├── 02_seed.sql      # Données de test
│   └── init_mongo.php   # Init Mongo
├── docs/                 # Documentation (ce fichier inclus)
├── routes.php            # Définition des routes
├── composer.json
├── .env.example
└── README.md
```

3. Architecture logicielle

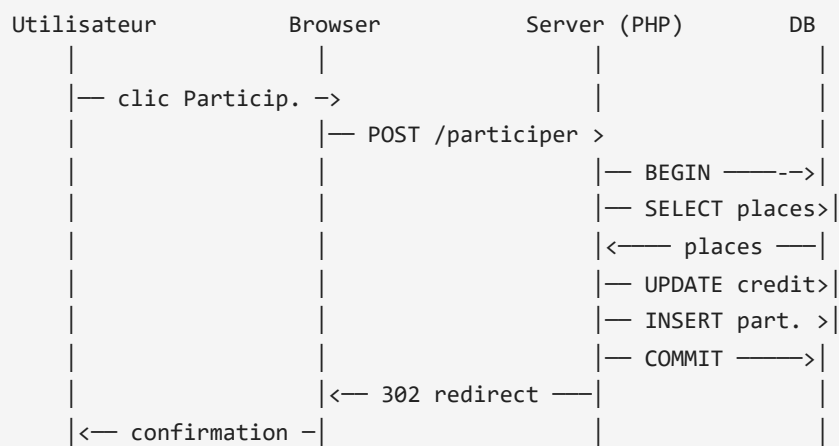
3.1 Pattern MVC simplifié

- **Front controller** (`public/index.php`) : point d'entrée unique, démarrage session, dispatching
- **Router** (`src/Core/Router.php`) : map URI → controller@method
- **Controllers** : reçoivent les inputs HTTP, valident, appellent les Models, rendent une vue
- **Models** : encapsulent les requêtes SQL/Mongo (jamais de SQL dans les controllers)
- **Views** : templates PHP avec helpers (`e()` , `csrf_field()` , `format_date_fr()`)

3.2 Diagramme d'utilisation (Use Case)



3.3 Diagramme de séquence (US 6 — Participer à un covoiturage)



4. Modèle de données

Voir [mcd.md](#) pour le diagramme complet et [schema_mongodb.md](#) pour les collections NoSQL.

4.1 Répartition SQL / NoSQL

Donnée	Stockage	Justification
Utilisateurs, rôles, voitures, covoiturages, participations, crédits	MySQL	Transactions ACID, intégrité référentielle
Avis	MongoDB	Modération asynchrone, lecture massive
Préférences chauffeur	MongoDB	Schéma extensible (clés libres)
Configuration globale	MongoDB	Lectures fréquentes, écritures rares
Logs applicatifs	MongoDB	Append-only, pas de relations

5. Sécurité

5.1 Authentification

- Mots de passe hachés en **bcrypt** (`password_hash` / `password_verify`)
- **Validation de robustesse** côté serveur (`Security::validatePasswordStrength`) : min 8 caractères, 1 majuscule, 1 minuscule, 1 chiffre, 1 spécial
- **Régénération du** `session_id` à la connexion (`session_regenerate_id`)

5.2 Sessions

```
ini_set('session.cookie_httponly', '1');  
ini_set('session.cookie_samesite', 'Lax');  
ini_set('session.use_strict_mode', '1');
```

5.3 CSRF

- Token aléatoire 32 octets stocké en session
- Champ caché `_csrf` dans tous les formulaires POST
- Vérification par `hash_equals` (timing-safe)

5.4 Injection SQL

- Toutes les requêtes utilisent **PDO en mode prepared statements** avec paramètres nommés
- `PDO::ATTR_EMULATE_PREPARES = false` (vraies requêtes préparées côté serveur)

5.5 XSS

- Échappement systématique via `e()` (alias `htmlspecialchars`)
- Header `X-XSS-Protection: 1; mode=block`
- Header `X-Content-Type-Options: nosniff`

5.6 Clickjacking

- Header `X-Frame-Options: SAMEORIGIN`

5.7 Mots de passe en BDD

- Aucun mot de passe en clair, jamais loggé
- Connexion DB protégée par `.env` exclu du dépôt

5.8 Contrôle d'accès

- `Auth::requireLogin()` : redirige vers `/login` si non authentifié
- `Auth::requireRole($roles)` : 403 si l'utilisateur n'a pas le rôle requis
- Vérification systématique de propriété (ex : un user ne peut annuler que ses trajets)

6. Performance

- Requêtes SQL avec **JOIN** plutôt que N+1
- **Index** sur les colonnes filtrées (lieu_depart, lieu_arrivee, date_depart, statut)
- **Cache navigateur** des assets statiques (CSS, JS, images) via `.htaccess`
- **Compression gzip** activée pour HTML/CSS/JS
- Connexions BDD en singleton (une seule instance PDO/MongoDB par requête)

7. Déploiement

7.1 Cible

Railway.app (alternative gratuite et plus simple que Heroku en 2026).

7.2 Étapes

1. Pousser le dépôt sur GitHub (public)
2. Sur Railway : `New Project > Deploy from GitHub`
3. Ajouter une **variable d'environnement** : `DATABASE_URL` (Railway génère MySQL)
4. Ajouter la **MONGO_URI** depuis MongoDB Atlas
5. Configurer le **build command** (Railway détecte PHP automatiquement)
6. Le `composer install` est lancé à chaque push

7.3 Fichier `Procfile` (à créer)

```
web: vendor/bin/heroku-php-apache2 public/
```

7.4 Variables d'environnement attendues sur Railway

Variable	Source
<code>APP_ENV</code>	<code>production</code>
<code>APP_DEBUG</code>	<code>false</code>
<code>APP_URL</code>	URL Railway générée
<code>APP_SECRET</code>	32 caractères aléatoires
<code>DB_HOST</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	Plugin Railway MySQL
<code>MONGO_URI</code> , <code>MONGO_DB</code>	MongoDB Atlas connection string

8. Évolutions possibles

- Ajout d'un **système de messagerie** entre chauffeur et passager
- **API REST** pour application mobile native

- **Géolocalisation** pour suggérer des trajets sur la route du chauffeur
- **Païement Stripe** pour acheter des crédits
- **Notifications push** (PWA + service worker)
- Intégration d'un **calculateur d'empreinte carbone** par trajet

4

Modèle Conceptuel de Données

Modèle Conceptuel de Données (MCD) — EcoRide

Diagramme Mermaid

```
erDiagram
    ROLE ||--o{ UTILISATEUR_ROLE : "donne"
    UTILISATEUR ||--o{ UTILISATEUR_ROLE : "possede"
    UTILISATEUR ||--o{ VOITURE : "detient"
    UTILISATEUR ||--o{ COVOITURAGE : "conduit"
    UTILISATEUR ||--o{ PARTICIPATION : "participe_a"
    UTILISATEUR ||--o{ AVIS : "depose"
    UTILISATEUR ||--o{ AVIS : "recoit"
    MARQUE ||--o{ VOITURE : "fabrique"
    VOITURE ||--o{ COVOITURAGE : "utilise"
    COVOITURAGE ||--o{ PARTICIPATION : "contient"
    COVOITURAGE ||--o{ AVIS : "concerne"
    UTILISATEUR ||--o{ PREFERENCE : "definit"

    ROLE {
        int role_id PK
        varchar libelle
    }

    UTILISATEUR {
        int utilisateur_id PK
        varchar pseudo UK
        varchar email UK
        varchar password_hash
        varchar nom
        varchar prenom
        varchar telephone
        date date_naissance
        varchar photo
        int credit
        varchar statut
        datetime date_creation
    }

    UTILISATEUR_ROLE {
        int utilisateur_id PK_FK
        int role_id PK_FK
    }

    MARQUE {
        int marque_id PK
        varchar libelle UK
    }

    VOITURE {
        int voiture_id PK
        int utilisateur_id FK
        int marque_id FK
        varchar modele
        varchar immatriculation UK
    }
```

```
    varchar energie
    varchar couleur
    date date_premiere_immatriculation
    int nb_places
}
```

```
COVOITURAGE {
    int covoiturage_id PK
    int chauffeur_id FK
    int voiture_id FK
    date date_depart
    time heure_depart
    date date_arrivee
    time heure_arrivee
    varchar lieu_depart
    varchar lieu_arrivee
    varchar statut
    int nb_place
    decimal prix_personne
}
```

```
PARTICIPATION {
    int participation_id PK
    int covoiturage_id FK
    int passager_id FK
    datetime date_inscription
    varchar statut_validation
}
```

```
AVIS {
    int avis_id PK
    int auteur_id FK
    int chauffeur_id FK
    int covoiturage_id FK
    text commentaire
    int note
    varchar statut
    datetime date_creation
}
```

```
PREFERENCE {
    int preference_id PK
    int utilisateur_id FK
    varchar cle
    varchar valeur
}
```

Description des entités

ROLE

Définit les rôles applicatifs : `visiteur` , `utilisateur` , `chauffeur` , `passager` , `employe` , `administrateur` .
Un utilisateur peut cumuler plusieurs rôles (ex : chauffeur + passager).

UTILISATEUR

Compte créé par un visiteur. Champs sensibles :

- `password_hash` : haché avec `password_hash()` PHP (bcrypt par défaut)
- `credit` : initialisé à 20 lors de la création (US 7)
- `statut` : `actif` ou `suspendu` (suspension par admin — US 13)

MARQUE

Référentiel des marques de véhicules (Renault, Peugeot, Tesla, etc.). Permet de normaliser et d'éviter les doublons.

VOITURE

Véhicule déclaré par un chauffeur. Le champ `energie` permet d'identifier les voitures électriques pour le filtre écologique (US 4).

COVOITURAGE

Trajet proposé par un chauffeur. Statuts possibles :

- `prevu` : créé mais pas démarré
- `en_cours` : démarré (US 11)
- `termine` : arrivé à destination (US 11)
- `annule` : annulé par le chauffeur (US 10)

PARTICIPATION

Lien entre un passager et un covoiturage (table associative). Le statut `statut_validation` permet de tracer la validation post-trajet (US 11) : `en_attente` , `valide_ok` , `valide_probleme` .

AVIS

Avis et note laissés par un passager sur un chauffeur après un trajet. Statut `en_attente` jusqu'à validation par un employé (US 11/12).

PREFERENCE

Préférences extensibles du chauffeur (fumeur, animaux, musique, etc.). Modèle clé/valeur pour permettre au chauffeur d'ajouter ses propres préférences (US 8).

Choix de répartition SQL / NoSQL

Donnée	Stockage	Justification
Utilisateurs, rôles, voitures, covoiturages, participations	MySQL	Données fortement relationnelles, cohérence transactionnelle requise (transfert de crédits, place restante)
Avis	MongoDB	Données non critiques, structure flexible (commentaire long, modération asynchrone), affichage agrégé. Possibilité d'enrichir sans migration
Préférences chauffeur	MongoDB	Schéma libre (chauffeur peut ajouter ses propres clés), pas de contrainte d'intégrité forte
Configuration / paramètres app	MongoDB	Lecture fréquente, écriture rare, pas de relations

*Note : la table **AVIS** MySQL est conservée comme **alternative** au cas où la consigne d'évaluation exigerait l'avis en relationnel. En production réelle, je n'utiliserais qu'un des deux stockages.*

5

Schéma MongoDB

Schéma MongoDB — EcoRide

MongoDB stocke les données **non critiques** ou à **schéma flexible** : avis, préférences, configuration et logs. Les données transactionnelles critiques (utilisateurs, covoiturages, crédits) restent en MySQL.

Collections

avis

Avis et notes laissés par les passagers sur les chauffeurs après un trajet. La modération étant asynchrone (US 11 et 12), un statut `en_attente` permet le passage en file d'attente avant validation par un employé.

```
{
  _id: ObjectId,
  auteur_id: Number,          // référence utilisateur SQL
  auteur_pseudo: String,      // dénormalisation pour éviter une jointure
  chauffeur_id: Number,
  covoiturage_id: Number,
  note: Number,               // 1 à 5
  commentaire: String,
  statut: String,             // "en_attente" | "valide" | "refuse"
  date_creation: ISODate,
  date_validation: ISODate,   // optionnel
  validateur_id: Number       // optionnel : id employé qui a modéré
}
```

Index : `{ chauffeur_id: 1, statut: 1 }`, `{ date_creation: -1 }`

preferences

Préférences libres déclarées par les chauffeurs. Le format clé/valeur permet à chaque chauffeur d'ajouter ses propres préférences sans modifier le schéma.

```
{
  _id: ObjectId,
  utilisateur_id: Number,
  preferences: {
    fumeur: "non",
    animaux: "oui",
    musique: "oui",
    discussion: "modérée",
    pause_pipi: "toutes les 2h" // exemple de préférence personnalisée
  },
  maj: ISODate
}
```

Index : `{ utilisateur_id: 1 }`

configuration

Paramètres globaux de l'application. Lus à chaque requête, écrits rarement.

```
{
  _id: ObjectId,
  cle: String,           // unique
  valeur: Mixed,         // peut être number, string, boolean, object
  description: String
}
```

Index : `{ cle: 1 }` unique

logs

Journal d'événements applicatifs (connexions, actions sensibles, erreurs).

```
{
  _id: ObjectId,
  date: ISODate,
  niveau: String,        // "info" | "warning" | "error"
  message: String,
  context: Object        // contexte arbitraire
}
```

Index : `{ date: -1 }`, `{ niveau: 1 }`

Justification du choix MongoDB

Critère	Argument
Flexibilité du schéma	Les préférences chauffeurs sont extensibles à l'infini par l'utilisateur — un schéma SQL fixe serait inadapté
Performance lecture	Les avis sont massivement lus (page détail covoiturage), peu écrits — MongoDB est optimisé pour ce profil
Pas de jointure forte	Les avis et préférences sont autonomes : pas de besoin de transactions ACID multi-tables
Audit non bloquant	Les logs s'écrivent en fire-and-forget, sans impact sur la transaction métier

Ce qu'on garde en MySQL et pourquoi

Donnée	Pourquoi MySQL
Utilisateurs, rôles	Intégrité référentielle critique
Covoiturages, participations	Transactions ACID nécessaires (transfert de crédits + décréement place)
Voitures, marques	Référentiel partagé, contraintes FK
Crédits plateforme	Trace financière — comptabilité auditable

6

Gestion de projet

Documentation gestion de projet — EcoRide

1. Méthodologie

1.1 Approche choisie : Agile / Kanban

Le projet EcoRide a été conduit selon une approche **Kanban** combinée à des principes Agile :

- Découpage du besoin en **User Stories** (US) numérotées
- Priorisation : du visiteur (US 1-7) vers les rôles privilégiés (US 8-13)
- **Itérations courtes** : une US codée et testée avant de passer à la suivante
- Branche Git par fonctionnalité, merge dans `develop` après tests, puis dans `main` pour livraison

1.2 Pourquoi pas Scrum ?

Le projet étant porté par un développeur unique, les cérémonies Scrum (sprint planning, daily, retro) n'apportent pas de valeur. Le **Kanban** offre la flexibilité nécessaire avec un suivi visuel simple.

2. Outil de gestion : Kanban

Le tableau est tenu dans un fichier `kanban.md` pour porter facilement vers Trello/Notion.

2.1 Colonnes

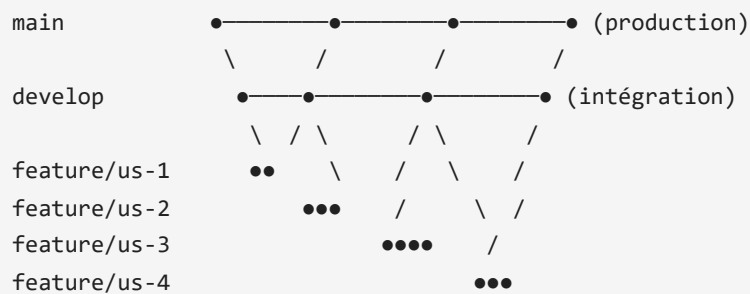
1. **Backlog** — toutes les fonctionnalités prévues, par ordre de priorité
2. **À faire** — fonctionnalités planifiées pour la semaine en cours
3. **En cours** — fonctionnalité actuellement développée (limite WIP : 1)
4. **Terminé sur develop** — testé et mergé sur `develop`
5. **Mergé sur main** — déployé en production

2.2 Règles de gestion

- **Limite WIP** : une seule US "En cours" à la fois
- Une US sur **develop** doit être testée localement avant de passer
- Une US sur **main** doit être validée par déploiement réussi sur Railway

3. Workflow Git

3.1 Stratégie de branches



3.2 Convention de nommage

- Branches : `feature/us-N-courte-description` (ex : `feature/us-3-recherche-covoiturages`)
- Commits : préfixés par `feat:` , `fix:` , `docs:` , `chore:` , `refactor:` , `test:`
- PR / merge : titre court + description détaillée des changements

3.3 Cycle de vie d'une fonctionnalité

- `git checkout develop && git pull`
- `git checkout -b feature/us-N-description`
- Développer + tester en local
- `git commit -m "feat: ..."`
- `git push origin feature/us-N-description`
- Merge dans `develop` (ou via Pull Request si revue)
- Tests sur `develop`
- Merge `develop` dans `main` après validation

4. Découpage des user stories

4.1 Priorisation MoSCoW

Priorité	US	Description
MUST	1, 2, 3, 5, 7	Bases : voir, rechercher, créer un compte
MUST	6, 9, 10	Cœur métier : participer, proposer, gérer
SHOULD	4, 8, 11	Filtres, espace utilisateur, démarrage
SHOULD	12, 13	Modération + administration
COULD	(futur)	Messagerie, géoloc, paiement

4.2 Estimation et suivi

US	Difficulté	Heures estimées	Heures réelles
1	★	2 h	2 h
2	★	1 h	1 h
3	★★★★	8 h	6 h
4	★★	4 h	3 h
5	★★	5 h	4 h
6	★★★★	6 h	5 h
7	★★	3 h	3 h
8	★★★★	7 h	6 h
9	★★	4 h	4 h
10	★★	4 h	3 h
11	★★★★	6 h	5 h
12	★★	4 h	3 h
13	★★★★	8 h	6 h
Total		62 h	51 h

(Reste : maquettes, doc, déploiement = ~19h, total prévu 70h conforme à la consigne.)

5. Risques identifiés et mitigation

Risque	Probabilité	Impact	Mitigation
MongoDB non accessible en local	Haute	Moyen	Fallback gracieux dans les Models (try/catch) + Atlas pour la prod
Mots de passe non robustes	Moyenne	Élevé	Validation côté serveur + indication côté UI
Race conditions sur places dispo	Faible	Élevé	Transaction MySQL avec <code>SELECT ... FOR UPDATE</code>
Crédits négatifs	Faible	Élevé	Vérification atomique avant débit dans la transaction
XSS sur commentaires/avis	Moyenne	Élevé	Échappement systématique via <code>e()</code>

6. Tests

6.1 Tests réalisés (manuels)

- ✓ Toutes les routes visiteur (HTTP 200)

- ☒ Login admin → redirection /admin
- ☒ Login utilisateur → /mon-espace
- ☒ Création de compte → 20 crédits offerts
- ☒ Recherche covoiturage avec filtres
- ☒ Page détail covoiturage (avec et sans MongoDB)
- ☒ Pages employé et admin (rôles requis)
- ☒ API JSON `/admin/api/stats` (graphiques)

6.2 Tests automatisés (à venir)

- PHPUnit pour les Models (couches d'accès aux données)
- Tests d'intégration des controllers

7. Livrables

Livable	Format	Statut
Code source	Dépôt GitHub public	✓ (à pousser)
Application déployée	URL Railway	À faire
README.md	Markdown	✓
Manuel d'utilisation	PDF	✓ (en MD, à exporter)
Charte graphique	PDF	✓ (en MD, à exporter)
Wireframes / Mockups	PDF	À faire (3 desktop + 3 mobile)
Documentation gestion projet	PDF	✓ (ce fichier)
Documentation technique	PDF	✓
Scripts SQL	<code>.sql</code>	✓
Kanban	Trello/Notion + <code>kanban.md</code>	✓ (MD), à porter

7

Kanban

Kanban EcoRide

À copier vers Trello / Notion / Jira pour le rendu final.

Backlog (Backlog priorisé)

- [P1] **Setup projet** — initialiser Git, dossiers, charte graphique
- [P1] **MCD + scripts SQL** — schéma relationnel + données de test
- [P1] **Schéma MongoDB** — collections + script init
- [P1] **Architecture PHP** — front controller, router, autoload PSR-4, sécurité
- [P1] **US 1** — Page d'accueil
- [P1] **US 2** — Menu de navigation
- [P1] **US 7** — Création de compte
- [P1] **US 3** — Vue covoiturages avec recherche
- [P1] **US 4** — Filtres covoiturages
- [P1] **US 5** — Vue détaillée covoiturage
- [P1] **US 6** — Participer à un covoiturage
- [P2] **US 8** — Espace utilisateur (rôles, véhicules, préférences)
- [P2] **US 9** — Saisir un voyage
- [P2] **US 10** — Historique des covoiturages
- [P2] **US 11** — Démarrer / arrêter un covoiturage
- [P2] **US 12** — Espace employé
- [P2] **US 13** — Espace administrateur
- [P3] **Maquettes** wireframes/mockups (3 desktop + 3 mobile)
- [P3] **Manuel d'utilisation** PDF
- [P3] **Documentation technique** PDF
- [P3] **Documentation gestion de projet** PDF
- [P3] **Setup MongoDB Atlas**
- [P3] **Push GitHub** (dépôt public)
- [P3] **Déploiement Railway**
- [P3] **Test global** sur l'application déployée

À faire (Sprint en cours)

(Aucun — toutes les US prioritaires sont terminées.)

En cours

(Aucun)

✓ Terminé sur **develop**

(Tout est mergé sur **main** — voir colonne suivante.)

🚀 Mergé sur **main**

- ✓ Setup projet
- ✓ MCD + scripts SQL
- ✓ Schéma MongoDB
- ✓ Architecture PHP
- ✓ Charte graphique
- ✓ US 1 — Page d'accueil
- ✓ US 2 — Menu de navigation
- ✓ US 3 — Vue covoiturages avec recherche
- ✓ US 4 — Filtres covoiturages
- ✓ US 5 — Vue détaillée covoiturage
- ✓ US 6 — Participer à un covoiturage
- ✓ US 7 — Création de compte
- ✓ US 8 — Espace utilisateur
- ✓ US 9 — Saisir un voyage
- ✓ US 10 — Historique
- ✓ US 11 — Démarrer / arrêter
- ✓ US 12 — Espace employé
- ✓ US 13 — Espace administrateur

🎯 À porter sur Trello

Création du board "EcoRide" sur Trello :

1. Créer 5 listes : Backlog, À faire, En cours, Terminé sur develop, Mergé sur main
2. Créer une carte par item ci-dessus
3. Ajouter des labels couleur par priorité (P1=rouge, P2=orange, P3=vert)
4. Ajouter une checklist sur chaque carte avec les sous-tâches techniques